Automatic Left Ventricle Segmentation

Using Deep Learning Models

Name: Md Jahid Hossain

MSc Data Science

University Of Hertfordshire

Comparative Evaluation of U-Net · Attention U-Net · Normal CNN · ViTSeg

Deep Learning for Medical Imaging | Cardiac SPECT Segmentation

Introduction

Clinical Context

Myocardial Perfusion refers to the flow of blood through the heart muscle.

Out of the four chambers of our heart, left ventricle is the strongest and the most important one. It distributes oxygenated blood throughout the body and sends blood into the main artery.

Left Ventricle (LV) segmentation is critical for quantifying myocardial perfusion deficits, measuring heart functions and detecting disease such as Cardiomyopathy or heart failures.

While manual delineation is time-consuming and observer-dependent, automated deep learning segmentation can enable reproducible, fast and scalable results.

Research Objective

Can lightweight, scratch-trained models achieve clinically relevant segmentation on 70×70 cardiac SPECT slices?

Previous Work

Ronneberger et al. (2015)

U-Net: Convolutional Networks for Biomedical Image Segmentation

Introduced the encoder-decoder architecture with skip connections — the foundational model tested here.

Oktay et al. (2018)

Attention U-Net: Learning Where to Look for the Pancreas

Extended U-Net with soft attention gates at skip connections; adopted here for LV boundary focus.

Long et al. (2015)

Fully Convolutional Networks (FCN) for Semantic Segmentation

Established FCN-style encoder-decoder baseline — the design behind NormalCNN tested here.

Dosovitskiy et al. (2020)

An Image is Worth 16×16 Words: Transformers for Image Recognition

Introduced Vision Transformers (ViT); adapted here as ViTSeg with 7×7 patches for 70×70 SPECT.

Betancur et al. (2018)

Deep Learning for Prediction of Obstructive Disease from Fast MPI

Demonstrated CNN applicability to myocardial perfusion SPECT clinical tasks.

What We Did — Research Plan

- Dataset: PhysioNet MPS Image Database v1.0 (96 usable SPECT patients)
- Task: 2D slice-level LV myocardial wall segmentation (binary mask)
- Train four architectures two from scratch and two pretrained weights
  - *U-Net · Attention U-Net · CNN · ViTSeg*
- Unified training pipeline: same loss, optimizer, augmentation for all models
  - *BCE + Dice loss · Adam · CosineAnnealingWarmRestarts · Early stopping*
- Evaluate on held-out test set (14 patients) using various metrics
  - *DSC · IoU · Sensitivity · Specificity ·*
- Identify the best model for clinical deployment based on DSC ≥ 0.80 threshold

Dataset — PhysioNet MPS Image Database v1.0

Dataset Overview

Link: https://www.physionet.org/content/myocardial-perfusion-spect/1.0.0/

Format: DICOM volumes + NIfTI masks (.nii)

Resolution: 70 × 70 pixels (native, no resize)

Total patients: 99

Excluded (BAD_UIDS): 3

- 2 — missing segmentation mask
- 1 — corrupted mask (~19 KB)

Usable patients: 96

Split (patient-level):

- Train: 68 patients
- Validation: 14 patients
- Test: 14 patients

Key Properties

Patient-level split → no data leakage

across train / val / test boundaries

Target: binary LV myocardial wall mask

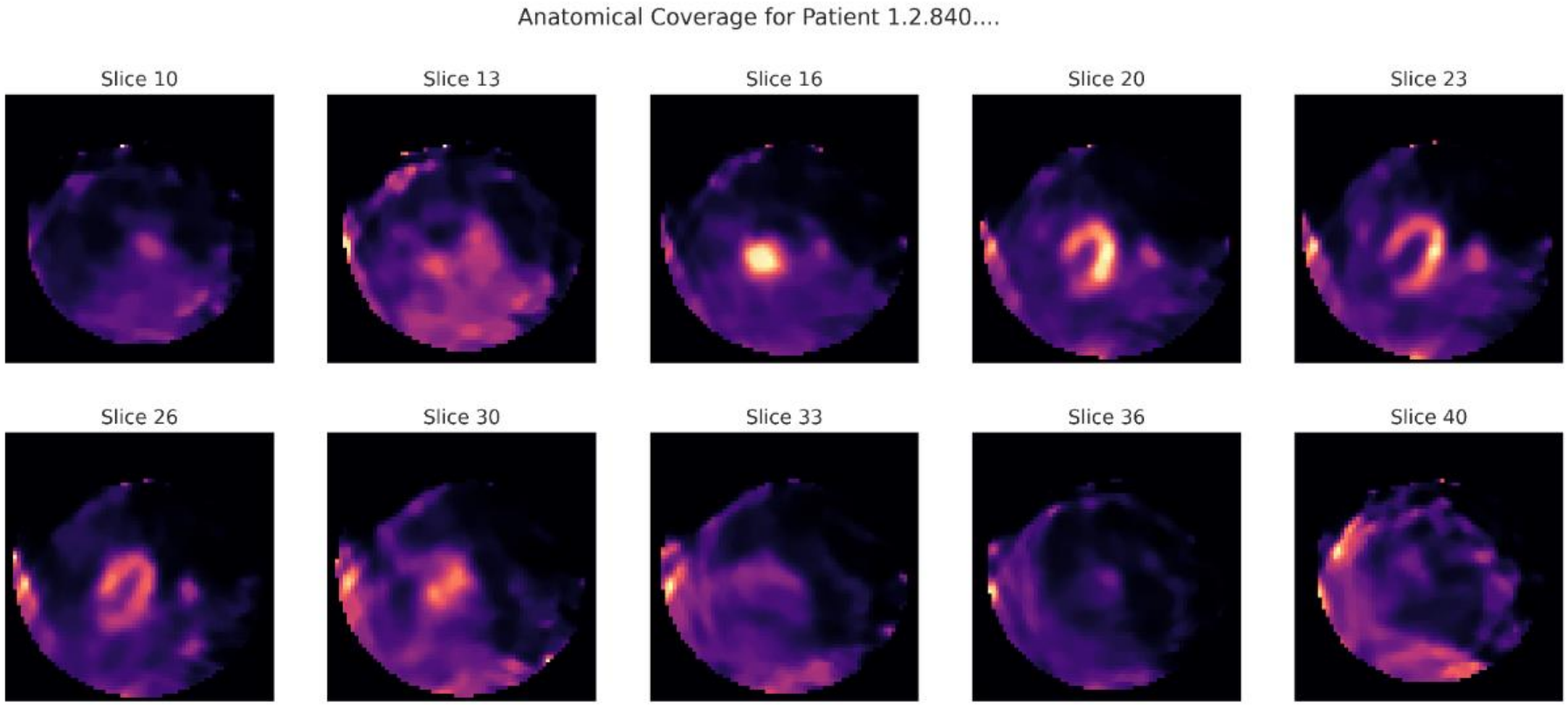
(mask binarised at threshold 0.5)

DICOM pixel arrays stacked into 3D

Volumes: NIfTI masks transposed to

match DICOM (z, y, x) orientation

Dataset — PhysioNet MPS Image Database v1.0



Ethical Implications

Patient Privacy

- PhysioNet MPS dataset is publicly available and de-identified.
- DICOM metadata does not contain direct patient identifiers.
- No additional IRB approval required for use of this public repository.

Algorithmic Fairness

- Dataset demographics not disclosed — performance may vary across subpopulations.
- Models should be validated on diverse multi-site SPECT datasets before deployment.

Clinical Safety

- Automated segmentation aids — not replaces — cardiologist review.
- DSC ≥ 0.80 is a guideline threshold; clinical sign-off remains mandatory.
- False negatives (missed LV tissue) carry higher clinical risk than false positives.

Reproducibility

- All code, model architectures, and hyperparameters are fully disclosed.
- Publicly available dataset ensures independent replication.

Exploratory Data Analysis (EDA)

96

Usable Patients

70×70

Native Resolution
(no resizing)

3

Excluded
(BAD_UIDs)

68 / 14 / 14

Train / Val / Test
(patient-level)

≥ 10 px

Min foreground
pixels per slice

5

Augmentation
transforms @ p=0.5

Key observation: SPECT images are low-resolution (70×70) and exhibit tracer uptake in adjacent organs (liver, bowel) that can bleed into the LV boundary — making precise delineation non-trivial.

Pre-processing Pipeline

1

DICOM Loading

All DICOM frames per study loaded via pydicom. Pixel arrays stacked into 3D volumes (z, y, x).

2

NIfTI Mask Alignment

Ground-truth masks loaded via nibabel. Transposed to (z, y, x) to match DICOM orientation.

3

Slice Filtering

Only slices with ≥ 10 foreground LV pixels retained (min_mask_pixels = 10). Eliminates near-empty frames.

4

Intensity Normalisation

Volume-level min-max normalisation to [0, 1] using v_min and v_max computed per 3D volume.

5

Mask Binarisation

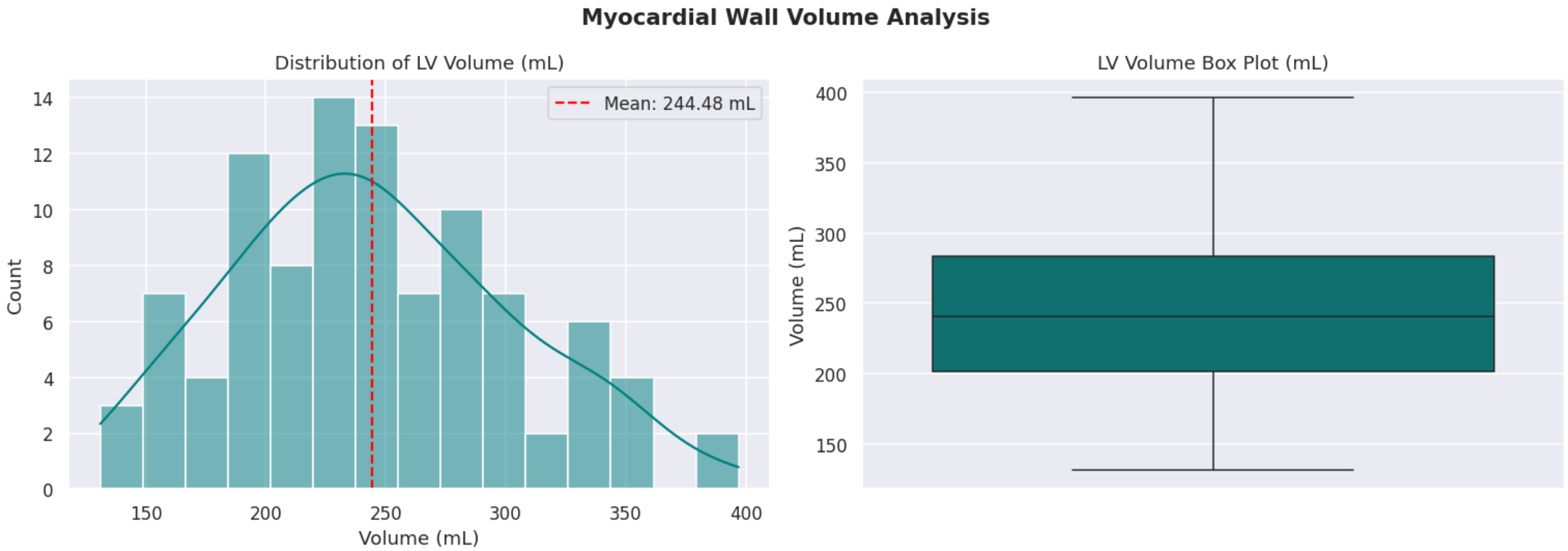
Mask values thresholded at 0.5 → binary {0, 1} segmentation targets.

6

Augmentation (train only)

Horizontal flip · Vertical flip · Rotation ±10° · Brightness ×[0.8,1.2] · Gaussian noise σ=0.02 — each at p=0.5

Dataset — PhysioNet MPS Image Database v1.0



Model Architectures (CNN Model)

```python
# ═══════════════════════════════════════════════════════════════════

class CNN(nn.Module):
    """Standard FCN-style CNN without skip connections."""
    def __init__(self, in_ch=1, out_ch=1):
        super().__init__()
        self.name = 'NormalCNN'
        self.enc = nn.Sequential(
            nn.Conv2d(in_ch, 32, 3, padding=1, bias=False), nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.Conv2d(32,    32, 3, padding=1, bias=False), nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Conv2d(32,  64, 3, padding=1, bias=False),  nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Conv2d(64,  64, 3, padding=1, bias=False),  nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2),
            nn.Conv2d(64,  128, 3, padding=1, bias=False), nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Conv2d(128, 128, 3, padding=1, bias=False), nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
        )
        self.dec = nn.Sequential(
            nn.Conv2d(128, 64, 3, padding=1, bias=False), nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Conv2d(64,  32, 3, padding=1, bias=False), nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
        )
        self.out = nn.Conv2d(32, out_ch, 1)

    def forward(self, x):
        feat = self.enc(x)
        feat = F.interpolate(feat, size=x.shape[2:], mode='bilinear',
        align_corners=True)
        return torch.sigmoid(self.out(self.dec(feat)))

normal_cnn  = NormalCNN().to(DEVICE)
params_ncnn = sum(p.numel() for p in normal_cnn.parameters() if p.requires_grad)
print(f'✅ NormalCNN   — Output: {normal_cnn(dummy).shape} | Params:
{params_ncnn:,}')
```

Model Architectures (Attention U-net Model)

```python
class AttentionUNet(nn.Module):
    def __init__(self, in_ch=1, out_ch=1, features=(16, 32, 64)):
        super().__init__()
        self.name = 'AttentionUNet'
        self.enc1 = DoubleConv(in_ch, features[0])
        self.enc2 = Down(features[0], features[1])
        self.enc3 = Down(features[1], features[2])
        self.bottleneck = Down(features[2], features[2] * 2)
        self.ag3  = AttentionGate(features[2]*2, features[2], features[2]//2)
        self.ag2  = AttentionGate(features[2],   features[1], features[1]//2)
        self.ag1  = AttentionGate(features[1],   features[0], features[0]//2)
        self.dec3 = Up(features[2]*2, features[2], features[2])
        self.dec2 = Up(features[2],   features[1], features[1])
        self.dec1 = Up(features[1],   features[0], features[0])
        self.out  = nn.Conv2d(features[0], out_ch, 1)

    def forward(self, x):
        s1 = self.enc1(x)
        s2 = self.enc2(s1)
        s3 = self.enc3(s2)
        b  = self.bottleneck(s3)
        x  = self.dec3(b,  self.ag3(b,  s3))
        x  = self.dec2(x,  self.ag2(x,  s2))
        x  = self.dec1(x,  self.ag1(x,  s1))
        return torch.sigmoid(self.out(x))


att_unet   = AttentionUNet().to(DEVICE)
params_att = sum(p.numel() for p in att_unet.parameters() if p.requires_grad)
print(f'✅ AttentionUNet — Output: {att_unet(dummy).shape} | Params:
{params_att:,}')
```

Model Architectures (U-net Models)

```python
# ═══════════════════════════════════════════════════════════
# MODEL 1 — Standard 2D U-Net (from scratch)
# Encoder: 16→32→64→128  Bottleneck: 256
# ═══════════════════════════════════════════════════════════

class UNet(nn.Module):
    def __init__(self, in_ch=1, out_ch=1, features=(16, 32, 64, 128)):
        super().__init__()
        self.name = 'UNet'
        self.enc1 = DoubleConv(in_ch, features[0])
        self.enc2 = Down(features[0], features[1])
        self.enc3 = Down(features[1], features[2])
        self.enc4 = Down(features[2], features[3])
        self.bottleneck = Down(features[3], features[3] * 2)
        self.dec4 = Up(features[3]*2, features[3], features[3])
        self.dec3 = Up(features[3],   features[2], features[2])
        self.dec2 = Up(features[2],   features[1], features[1])
        self.dec1 = Up(features[1],   features[0], features[0])
        self.out  = nn.Conv2d(features[0], out_ch, 1)

    def forward(self, x):
        s1 = self.enc1(x)
        s2 = self.enc2(s1)
        s3 = self.enc3(s2)
        s4 = self.enc4(s3)
        b  = self.bottleneck(s4)
        x  = self.dec4(b,  s4)
        x  = self.dec3(x,  s3)
        x  = self.dec2(x,  s2)
        x  = self.dec1(x,  s1)
        return torch.sigmoid(self.out(x))


unet       = UNet().to(DEVICE)
dummy      = torch.randn(2, 1, 70, 70).to(DEVICE)
out        = unet(dummy)
params_unet = sum(p.numel() for p in unet.parameters() if p.requires_grad)
print(f'✅ UNet — Output: {out.shape} | Params: {params_unet:,}')
```

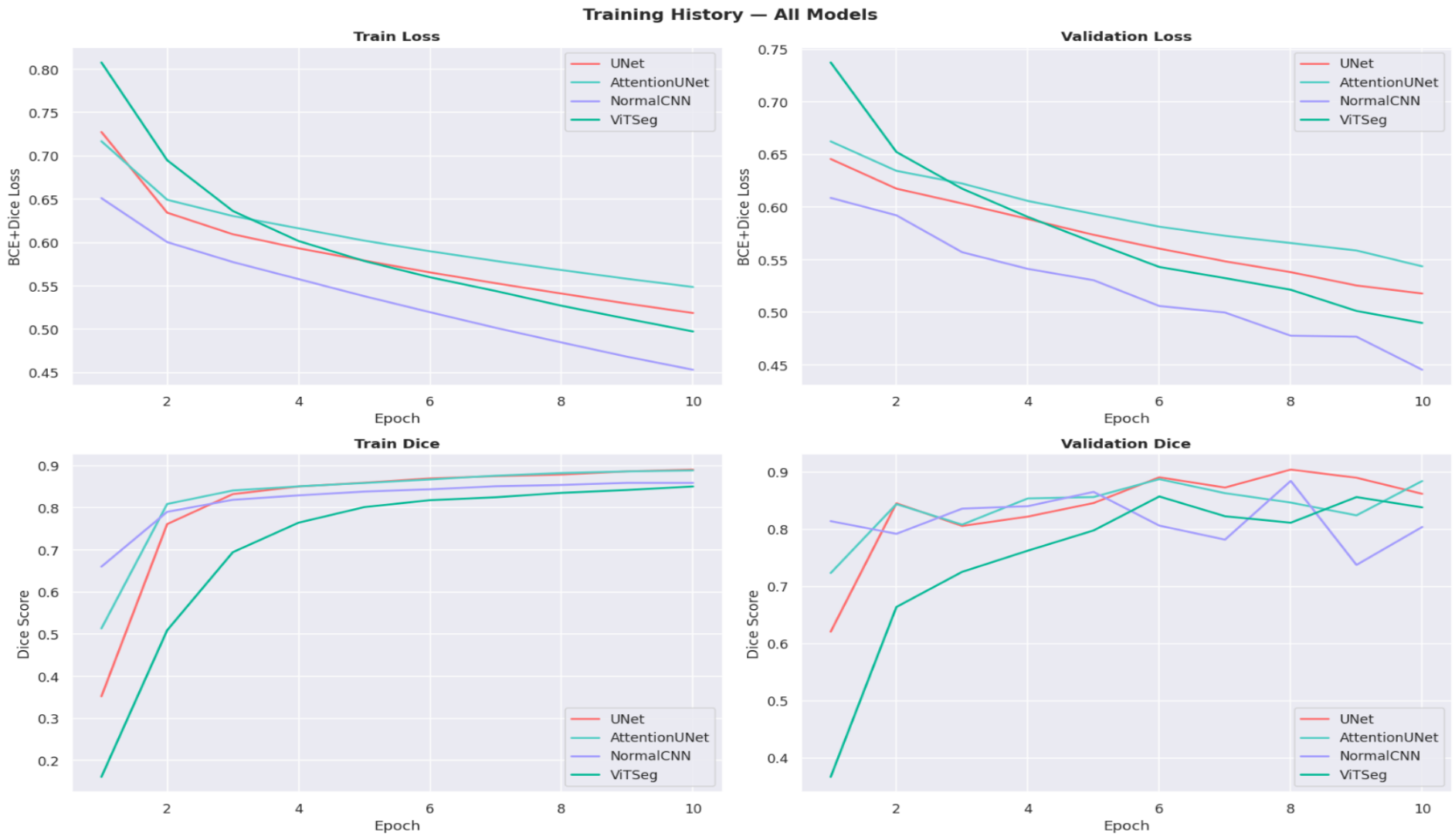
Model Architectures (Vit Models)

```python
class ViTSeg(nn.Module):

    def __init__(self, img_size=70, patch_size=7, in_ch=1, out_ch=1,
                 embed_dim=128, depth=4, n_heads=4, dec_ch=32):
        super().__init__()
        self.name        = 'ViTSeg'
        self.patch_size  = patch_size
        self.dec_ch      = dec_ch
        n_patches        = (img_size // patch_size) ** 2
        self.patch_embed = PatchEmbed(img_size, patch_size, in_ch, embed_dim)
        self.cls_token   = nn.Parameter(torch.zeros(1, 1, embed_dim))
        self.pos_embed   = nn.Parameter(torch.zeros(1, n_patches + 1,
        embed_dim))
        self.blocks      = nn.Sequential(
            *[TransformerBlock(embed_dim, n_heads) for _ in range(depth)])
        self.norm        = nn.LayerNorm(embed_dim)
        # Decoder: project each patch token → p×p spatial pixels
        self.dec_proj    = nn.Linear(embed_dim, patch_size * patch_size *
        dec_ch)
        self.dec_conv    = nn.Sequential(
            nn.Conv2d(dec_ch, dec_ch, 3, padding=1, bias=False),
            nn.BatchNorm2d(dec_ch), nn.ReLU(inplace=True),
            nn.Conv2d(dec_ch, out_ch, 1),
        )
        nn.init.trunc_normal_(self.pos_embed, std=0.02)
        nn.init.trunc_normal_(self.cls_token, std=0.02)

    def forward(self, x):
        B, _, H, W = x.shape
        p = self.patch_size
        tokens, gh, gw = self.patch_embed(x)                  # (B, N, E)
        cls    = self.cls_token.expand(B, -1, -1)
        tokens = torch.cat([cls, tokens], dim=1) + self.pos_embed
        tokens = self.norm(self.blocks(tokens))
        patch_tokens = tokens[:, 1:]                           # (B, N, E)
        # Pixel-shuffle decode
        feat = self.dec_proj(patch_tokens)                     # (B, N, p*p*C)
        feat = feat.view(B, gh, gw, p, p, self.dec_ch)
        feat = feat.permute(0, 5, 1, 3, 2, 4).contiguous()    # (B,C,gh,p,gw,p)
        feat = feat.view(B, self.dec_ch, gh * p, gw * p)      # (B,C,H,W)
        feat = F.interpolate(feat, size=(H, W), mode='bilinear',
        align_corners=True)
```

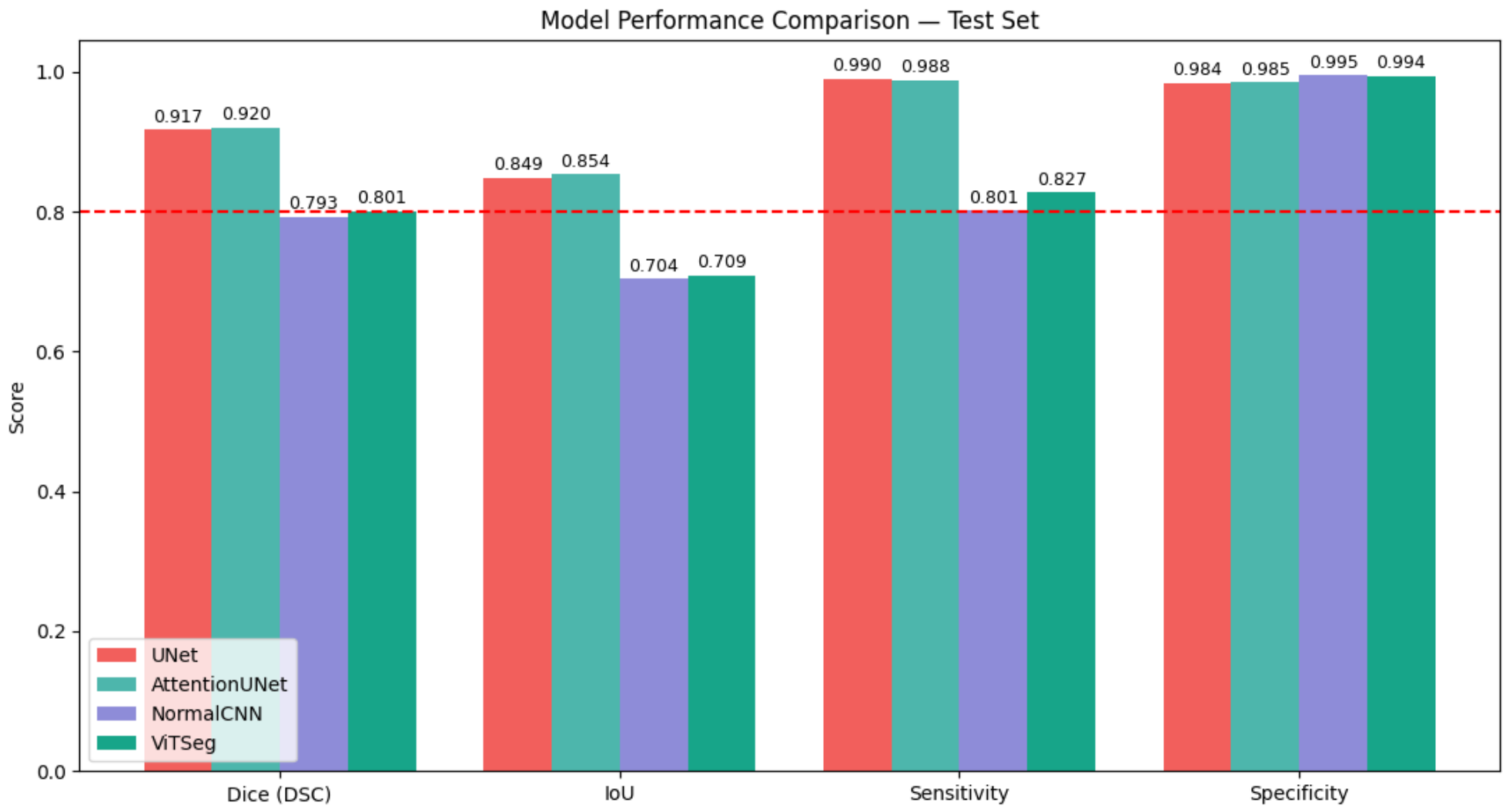
Model Results — Test Set Performance

Model	Phase	DSC ↑	IoU ↑	Sens. ↑	Spec. ↑	Params ↓	Latency (ms)
Attention U-Net	Hyper-Tuned	**0.9204**	0.8536	0.9879	0.9849	490,914	15.01
	Test Set	0.8816	0.7923	0.9743	0.9901	—	—
U-Net	Hyper-Tuned	0.9174	0.8486	0.9896	0.9840	1,942,289	16.61
	Test Set	0.8931	0.8097	0.9191	0.9943	—	—
ViTSeg	Test Set	0.8007	0.7093	0.8272	0.9936	—	—
Normal CNN	Test Set	0.7928	0.7040	0.8012	0.9949	—	—

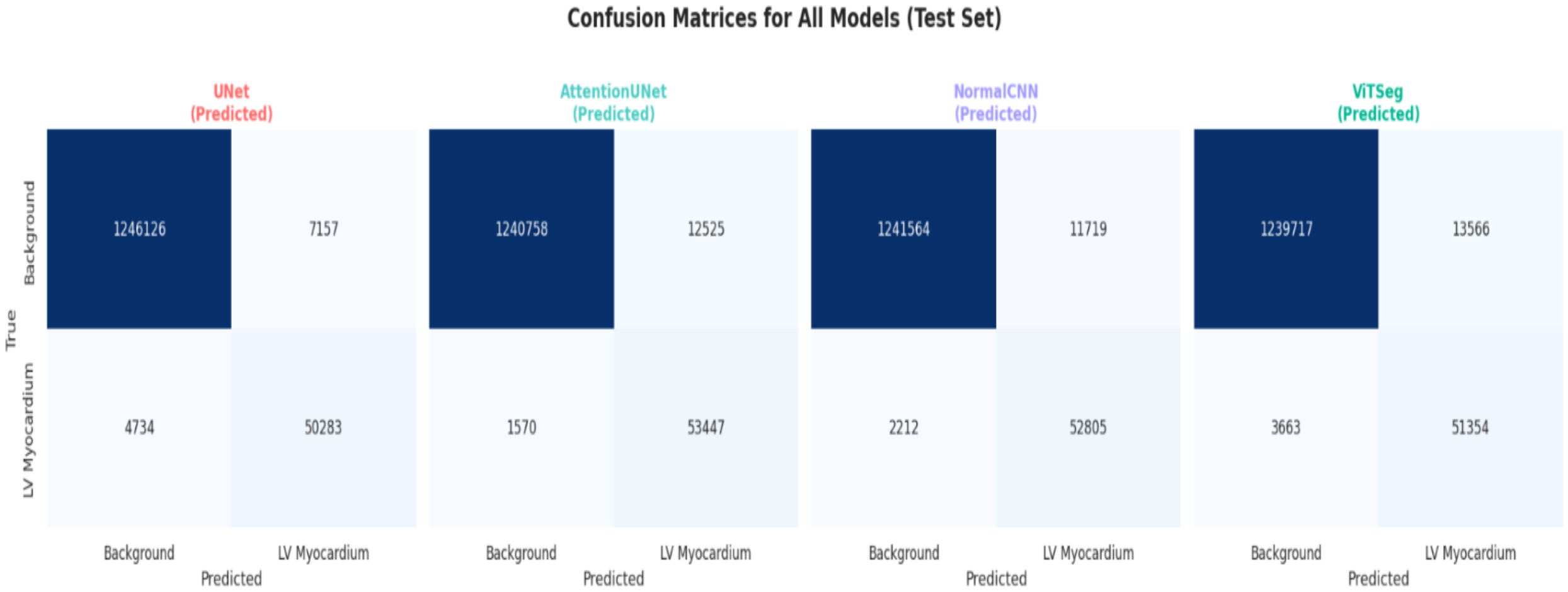
Model Architectures (4 Models)



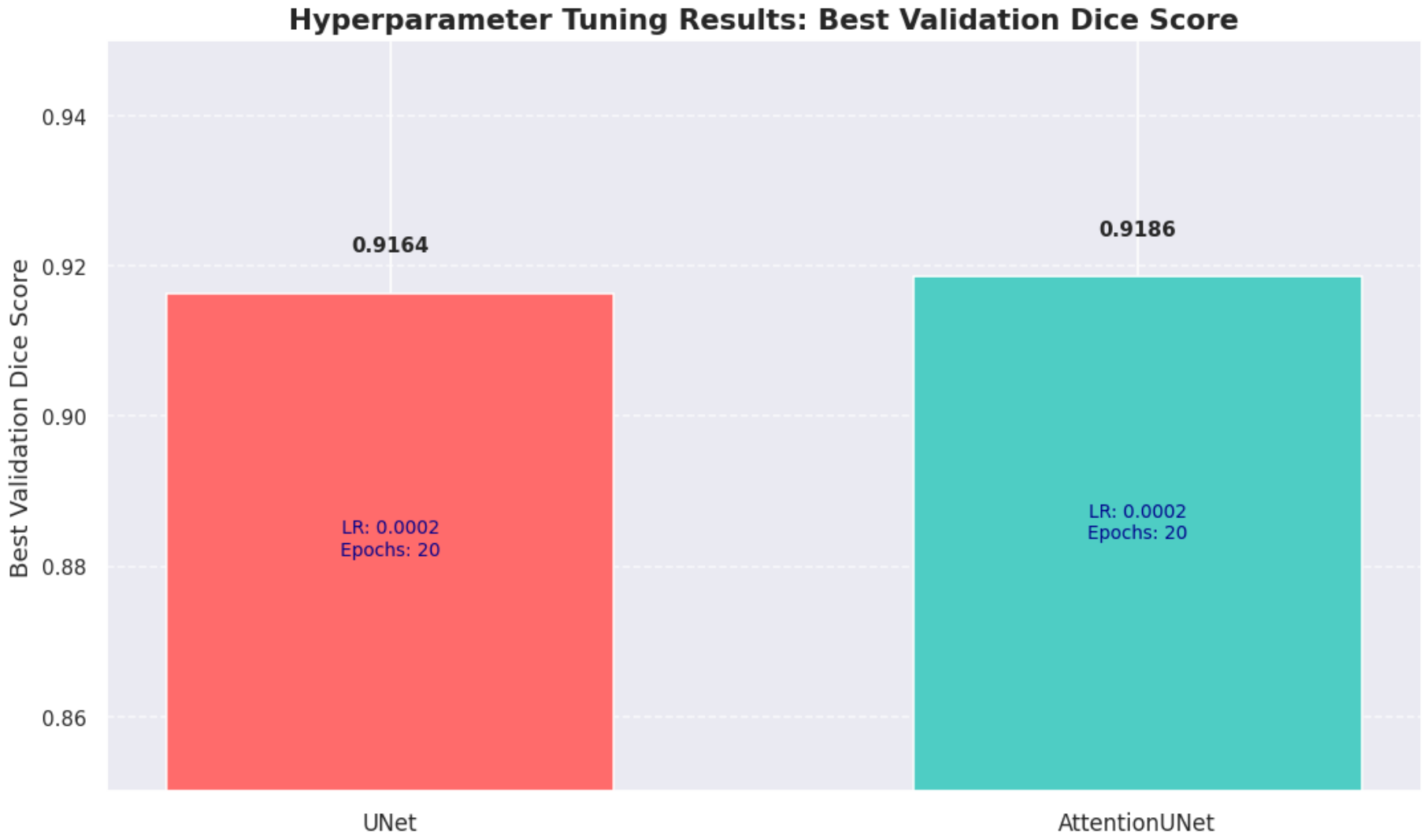
Model Architectures (4 Models)



Model Architectures (4 Models)



Model Architectures (4 Models)



Analysis & Discussion

Performance Leader

Attention U-Net achieved the highest overall performance with peak Hyper-Tuned DSC of 0.9204, final Test DSC of 0.8816, and best clinical Sensitivity (0.9743) on the test set.

Hyper-Tuning Success

Hyper-parameter tuning pushed Attention U-Net to a validation DSC of 0.9204, with strong stability on the final test set (0.8816), confirming the model generalises well.

Boundary Accuracy

While Attention U-Net leads in overlap, the standard U-Net achieves the tightest contour with HD = 2.22 px and a Test DSC of 0.8931, making it a robust baseline when geometric precision is prioritised.

Threshold Benchmarking

U-Net achieves best HD = 2.22 px (tightest boundary) while Attention U-Net leads on DSC. Skip connections + bilinear decoder precision favour U-Net for geometric accuracy.

Clinical Threshold: 3 of 4 Models

Attention U-Net, U-Net, and ViTSeg all exceed DSC ≥ 0.80. Normal CNN (0.7928) narrowly misses — skip connections appear necessary to consistently reach the clinical threshold on this 70×70 SPECT dataset.

Summary & Future work

Top Performance & Efficiency

Attention U-Net achieved peak Hyper-Tuned DSC of 0.9204 and final Test DSC of 0.8816, with best clinical Sensitivity (0.9743) — using only ~25% of the parameters (490,914 vs 1,942,289) at just 15.01 ms/slice.

Boundary Accuracy

Standard U-Net achieves the tightest contour with HD = 2.22 px (Test DSC 0.8931), remaining a robust baseline when geometric precision is prioritised.

Clinical Threshold

3 of 4 models (Attention U-Net, U-Net, ViTSeg) exceed DSC ≥ 0.80. Normal CNN (0.7928) narrowly misses — skip connections appear necessary on this 70×70 SPECT dataset.

Future Work — 3D Volumetric U-Net

Transition from 2D slice segmentation to 3D convolution kernels to exploit inter-slice spatial context and volumetric continuity.

Future Work — Paired Perfusion Analysis

Develop a dual-stream framework for stress-rest paired SPECT analysis to improve diagnostic specificity of perfusion defect detection.

Future Work — Multi-site Validation

Validate models on diverse multi-centre SPECT datasets to assess generalisability across scanner types and patient demographics before clinical deployment.

References

[1] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI 2015. Springer, LNCS 9351, pp. 234–241.

[2] Oktay, O., et al. (2018). Attention U-Net: Learning Where to Look for the Pancreas. MIDL 2018. arXiv:1804.03999.

[3] Long, J., Shelhamer, E., & Darrell, T. (2015). Fully Convolutional Networks for Semantic Segmentation. CVPR 2015, pp. 3431–3440.

[4] Dosovitskiy, A., et al. (2020). An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale. ICLR 2021. arXiv:2010.11929.

[5] Betancur, J., et al. (2018). Deep Learning for Prediction of Obstructive Disease from Fast Myocardial Perfusion SPECT. JACC: Cardiovascular Imaging, 11(11), 1654–1663.

[6] Goldberger, A. L., et al. (2000). PhysioBank, PhysioToolkit, and PhysioNet. Circulation, 101(23), e215–e220. DOI: 10.13026/ce2z-dw74.

[7] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS 2019.

[8] van der Walt, S., et al. (2014). scikit-image: Image processing in Python. PeerJ 2:e453. (hausdorff_distance implementation)